EY Techathon Retail Agent

Complete Technical Documentation

Multi-Agent AI Retail Assistant

Semi-Finals Presentation Guide

Tech Stack: Python FastAPI | LangGraph | OpenAI GPT-4o-mini | SQLite | React

February 2026

Table of Contents

- 1. Architecture Overview
- 2. Database Layer
- 3. Models (Data Schema)
- 4. CRUD Operations (Universal Pattern)
- 5. Agent Tools (All 12 Tools - Detailed)
- 6. Agent Graph & Orchestration (LangGraph)
- 7. All API Endpoints (Detailed)
- 8. WebSocket Real-time Communication
- 9. Chat Database & History
- 10. Store Kiosk Flow
- 11. WhatsApp Integration
- 12. Security
- 13. Data Import Pipeline
- 14. Migration Scripts
- 15. Configuration
- 16. Key Design Patterns Summary

1. Architecture Overview

This is a multi-agent AI retail assistant built for an omnichannel shopping experience. The system uses an LLM (GPT-4o-mini) as a Master Orchestrator that decides which specialized agent tools to call based on user conversation. The architecture follows a reactive agentic loop powered by LangGraph.

Technology Stack

- Backend: Python FastAPI + LangGraph + OpenAI GPT-4o-mini
- Frontend: React + Vite
- Database: SQLite via SQLModel/SQLAlchemy
- Agent Framework: LangGraph (StateGraph with tool calling)
- Real-time: WebSocket for timeline updates
- Channels: Web Chat, Store Kiosk, WhatsApp

Core Agent Loop

```
START
  |
  v
[Agent Node (LLM)] --> decides --> [Tool Node (executes tools)]
  ^                                        |
  |________________________________________|   (loop)
  |
  v
 END (when LLM has no more tool calls)
```

The LLM receives the user message plus all 12 tool definitions. It decides which tool(s) to call, the Tool Node executes them, results go back to the LLM, and the LLM either calls more tools or generates a final text response to the user.

2. Database Layer

File: backend/database.py

Database Configuration

```
DATABASE_URL = "sqlite:///./retail_agent.db"
engine = create_engine(
    DATABASE_URL,
    echo=True,
    connect_args={"check_same_thread": False}
)
```

Key Functions

Function	Purpose
init_db()	Creates all tables from SQLModel metadata
get_session()	Yields a session (FastAPI dependency injection)
get_db_session()	Returns a session directly (for agent tools)

SQLite Pragma: On every connection, PRAGMA foreign_keys=ON is enforced via SQLAlchemy event listener _set_sqlite_pragma to ensure foreign key constraints are respected.

Dual Database Architecture

The project uses TWO separate SQLite databases:
1. retail_agent.db - Main business database (products, customers, orders, cart)
2. chat_history.db - Conversation persistence database (chat messages, sessions)

This separation keeps business data clean from transient chat data.

3. Models (Data Schema)

File: backend/models.py

All models use SQLModel (which combines SQLAlchemy + Pydantic). Each model maps to a database table.

Model	Primary Key	Description
Customer	customer_id	Customer profiles (name, email, address, loyalty_points, customer_type, tier)
Product	product_id	Product catalog (name, category, gender, price, quantity, availability, store_ids, season, festival, image, loyalty_points)
Store	store_id	Physical store locations (name, address, latitude, longitude)
OrderHistory	transaction_id	Past orders (customer_id, product_id, dates, amount, mode, payment_method)
StoreKeeperTransaction	id	Store keeper transaction logs
AdminDashboardData	id	Pre-computed admin analytics data
Cart	order_id	Shopping cart items (customer_id, product_id, name, price, quantity, size, status)
Payload	id	Stores JSON payloads per customer
Distance	id	Customer-to-store distance mappings
Feedback	id	Customer feedback after purchase

Key Model Relationships

- Cart.customer_id -> Customer.customer_id (Foreign Key)
- Cart.product_id -> Product.product_id (Foreign Key)
- OrderHistory.customer_id -> Customer.customer_id
- OrderHistory.product_id -> Product.product_id
- Product.store_ids -> Comma-separated Store.store_id values

4. CRUD Operations (Universal Pattern)

File: backend/crud.py

The project uses a Universal CRUD pattern - one generic class UniversalCRUD that works with ANY SQLModel table, reducing boilerplate massively.

UniversalCRUD Class

```
class UniversalCRUD:
    def __init__(self, model: Type[SQLModel], primary_key: str = "id"):
        self.model = model
        self.primary_key = primary_key
```

All Methods

Method	Signature	What It Does
create	(session, data: dict) -> SQLModel	Insert one record into table
bulk_create	(session, data_list) -> List	Insert multiple records at once
get_by_id	(session, record_id) -> Optional	Fetch single record by primary key
get_all	(session, skip, limit) -> List	Paginated fetch of all records
filter	(session, filters, operators, ...)	Dynamic filtering with operators (=, >, <, like, in)
update	(session, record_id, data)	Update specific fields by primary key
delete	(session, record_id) -> bool	Delete record by primary key
count	(session, filters) -> int	Count records matching filters
exists	(session, filters) -> bool	Check if any records match filters
search	(session, fields, term) -> List	Full-text search across multiple fields

Pre-configured CRUD Instances

```
customer_crud = UniversalCRUD(Customer, primary_key="customer_id")
product_crud = UniversalCRUD(Product, primary_key="product_id")
store_crud = UniversalCRUD(Store, primary_key="store_id")
order_history_crud = UniversalCRUD(OrderHistory, primary_key="transaction_id")
cart_crud = UniversalCRUD(Cart, primary_key="order_id")
payload_crud = UniversalCRUD(Payload, primary_key="id")
```

Specialized CRUD Classes

These extend UniversalCRUD with model-specific methods:

- ProductCRUD: adds search_products(session, query) - searches by name, category, gender, season
- CustomerCRUD: adds search_customers(session, query) - searches by name, email, customer_id
- CartCRUD: adds get_customer_cart(session, customer_id) - gets active cart items

5. Agent Tools (All 12 Tools - Detailed)

File: backend/intelligent_agent_graph.py

All tools are decorated with @tool from LangChain. The LLM sees their docstrings and decides when to call them. Each tool performs specific DB operations and returns JSON responses.

Tool 1: customer_info_agent

Purpose

Fetches customer profile information and order history from the database.

Parameters

- customer_id: str - The customer ID to look up
- info_type: str = "profile" - Either "profile" or "orders"

DB Operations

```
# For profile:
customer = customer_crud.get_by_id(session, customer_id)
# Returns: name, email, loyalty_points, tier, address, customer_type

# For orders:
orders = order_history_crud.filter(session, {"customer_id": customer_id})
# Returns: list of past orders with dates, amounts, payment methods
```

When LLM Calls It

User says: "Who am I?", "Show my profile", "What are my loyalty points?", "Show my orders"

Return Format

```
{
  "customer_name": "John Doe",
  "email": "john@example.com",
  "loyalty_points": 1500,
  "tier": "Gold",
  "address": "123 Main St",
  "customer_type": "Premium"
}
```

Tool 2: inventory_agent

Purpose

The most complex tool. Searches product inventory using multi-criteria filtering, OR checks store availability for a specific product (offline flow).

Parameters

- search_query: str = "" - Free-text search like "hoodies", "red dress", "products between 1500-2000"
- product_id: str = "" - Specific product ID for store availability check (offline flow)
- customer_id: str = "" - Used to filter out already-purchased and already-shown products

DB Operations - Search Mode

```
# Load all products
all_products = product_crud.get_all(session)

# Filter by category, gender, season using fuzzy matching
# Parse price range from query via _parse_price_band_from_query()
# Filter out already purchased products
purchased = get_customer_purchased_product_ids(session, customer_id)

# Filter out already shown products (in-memory dedup)
_SHOWN_PRODUCT_IDS_BY_CUSTOMER_AND_QUERY[(customer_id, query)].add(pid)
```

DB Operations - Store Availability Mode

```
# Get product details
product = product_crud.get_by_id(session, product_id)

# Parse store_ids (comma-separated)
store_ids = product.store_ids.split(",")

# Fetch each store
for sid in store_ids:
    store = store_crud.get_by_id(session, sid)
    # Build store list with name, address, lat/long
```

Price Band Parsing Logic

```
def _parse_price_band_from_query(query):
    # "between 1500 and 2000" -> min=1500, max=2000
    # "above 5000"            -> min=5000
    # "under 1000"            -> max=1000
    # "1500-2000"             -> min=1500, max=2000
```

"Show More" Deduplication Feature

```
# In-memory tracking per (customer_id, query) pair
```

```
_SHOWN_PRODUCT_IDS_BY_CUSTOMER_AND_QUERY: dict[tuple, set] = {}

# When user says "show me more hoodies":
# Previously shown product IDs are excluded from results
# New products are added to the tracking set
```

Tool 3: recommendation_agent

Purpose

Formats raw inventory results into beautiful product recommendation cards for the frontend. This is a presentation/formatting layer - no direct DB operations.

Parameters

- inventory_data: str - JSON string from inventory_agent output

DB Operations

None directly. Works on data passed from inventory_agent.

Returns

```
{
  "products": [
    {
      "product_id": "P001",
      "product_name": "Blue Hoodie",
      "price": 1500,
      "category": "Hoodies",
      "image_reference": "hoodie.jpg",
      "online_available": true,
      "offline_available": true,
      "add_to_cart_button": true
    }
  ]
}
```

Tool Chaining Rule

The LLM is instructed to ALWAYS chain: inventory_agent -> recommendation_agent

Tool 4: upsell_agent

Purpose

Autonomous upselling - suggests accessory products after the user views their cart. Randomly picks from accessory categories and generates a search query.

Parameters

- customer_id: str

Logic

```
accessory_categories = ["belts", "shoes", "watches", "wallets"]
```

```
selected = random.choice(accessory_categories)
search_query = f"show me {selected}"
# LLM then calls inventory_agent with this query
```

When Called

Automatically triggered after view_cart_agent completes. This is an intelligent upselling mechanism.

Tool 5: add_to_cart_agent

Purpose

Adds a product to the customer shopping cart with stock validation and duplicate detection.

Parameters

- product_id: str - Product to add
- quantity: int = 1 - How many units
- customer_id: str - Which customer
- size: str = "M" - Size selection

DB Operations (Step by Step)

```
# Step 1: Validate product exists and check stock
product = product_crud.get_by_id(session, product_id)
if product.quantity < requested_quantity:
    return "Insufficient stock"

# Step 2: Check if item already in cart (duplicate detection)
existing = cart_crud.filter(session, {
    "customer_id": customer_id,
    "product_id": product_id
})

# Step 3a: If exists -> increment quantity
if existing:
    cart_crud.update(session, existing.order_id, {
        "quantity": existing.quantity + new_quantity
    })

# Step 3b: If new -> create new cart entry
else:
    order_id = f"ORD-{timestamp}-{random_hex}"
    cart_crud.create(session, {
        "order_id": order_id,
        "customer_id": customer_id,
        "product_id": product_id,
        "product_name": product.product_name,
        "price": product.price,
        "quantity": quantity,
        "size": size,
        "status": "active"
    })
```

Validation Logic

- Checks stock availability before adding
- Prevents adding more than available quantity

- Handles duplicate detection (same product -> quantity increment)

Tool 6: view_cart_agent

Purpose

Shows all items currently in the customer cart with totals and loyalty points info.

Parameters

- customer_id: str

DB Operations

```
# Get all active cart items
cart_items = cart_crud.filter(session, {
    "customer_id": customer_id,
    "status": "active"
})

# For each item, fetch full product details
for item in cart_items:
    product = product_crud.get_by_id(session, item.product_id)

# Get customer loyalty points
customer = customer_crud.get_by_id(session, customer_id)
```

Returns

```
{
  "cart_items": [...],
  "cart_total": 3500,
  "available_loyalty_points": 1500,
  "item_count": 3
}
```

Tool 7: remove_from_cart_agent

Purpose

Removes a specific item from the customer cart.

DB Operations

```
# Find the cart item
cart_item = cart_crud.filter(session, {
    "customer_id": customer_id,
    "product_id": product_id
})

# Delete it
cart_crud.delete(session, cart_item.order_id)
```

Tool 8: choose_channel_agent

Purpose

Handles online vs offline purchase channel selection at checkout time.

Parameters

- customer_id: str
- channel: str = "" - "online" or "offline"

DB Operations

```
# Verify cart has items
cart_items = cart_crud.filter(session, {
    "customer_id": customer_id,
    "status": "active"
})
```

Flow Routing

- If "online" -> proceeds to payment_agent flow
- If "offline" -> triggers inventory_agent for store availability
- If empty -> asks user to choose between online/offline

Tool 9: payment_agent

Purpose

Two-phase payment processing. Phase 1 shows payment options, Phase 2 processes the payment.

Parameters

- customer_id: str
- payment_method: str = "" - Empty for Phase 1, or "upi"/"credit card"/"debit card"/"net banking"

Phase 1: Show Payment Options (no payment_method)

```
# Get cart for total calculation
cart_items = cart_crud.filter(session, {
    "customer_id": customer_id,
    "status": "active"
})
# Returns available payment methods with cart total
```

Phase 2: Process Payment (with payment_method)

```
# Get cart items
cart_items = cart_crud.filter(session, {
```

```
    "customer_id": customer_id,
    "status": "active"
})

# For EACH cart item:
for item in cart_items:
    # 1. Deduct product stock
    deduct_product_quantity(product_id, quantity)
    # -> product_crud.get_by_id() then
    # -> product_crud.update(session, pid, {"quantity": new_qty})

    # 2. Create order history record
    order_history_crud.create(session, {
        "transaction_id": f"ORD-{timestamp}-{random}",
        "customer_id": customer_id,
        "product_id": product_id,
        "amount": price * quantity,
        "mode": "online",
        "payment_method": payment_method
    })

    # 3. Mark cart item as completed
    cart_crud.update(session, item.order_id, {"status": "completed"})
```

Critical Flow Rule (from System Prompt)

Call payment_agent FIRST TIME with only customer_id -> shows options
Call payment_agent SECOND TIME with payment_method -> processes payment
After SECOND call returns "Payment Successful", IMMEDIATELY call fulfillment_agent

Tool 10: fulfillment_agent

Purpose

Handles order fulfillment for both online delivery and offline store pickup. Different DB operations for each mode.

Parameters

- purchase_type: str = "online" - "online" or "offline"
- store_name: str = "" - For offline, which store for pickup
- customer_id: str | None
- order_id: str | None

Online Flow DB Operations

```
# Get customer address for delivery
customer = customer_crud.get_by_id(session, customer_id)

# Get completed order items
cart_items = cart_crud.filter(session, {
    "customer_id": customer_id,
    "status": "completed"
})
# Returns delivery confirmation with estimated date
```

Offline Flow DB Operations

```
# Get active cart items
cart_items = cart_crud.filter(session, {
    "customer_id": customer_id,
    "status": "active"
})

# For each item:
for item in cart_items:
    # 1. Deduct stock
    deduct_product_quantity(product_id, quantity)

    # 2. Create order with mode="offline"
    order_history_crud.create(session, {
        ..., "mode": "offline"
    })

    # 3. Mark completed
    cart_crud.update(session, item.order_id, {"status": "completed"})
```

Flow Differences

- Online: payment_agent -> fulfillment_agent -> loyalty_agent -> post_purchase_agent

- Offline: inventory_agent (stores) -> fulfillment_agent -> END (no loyalty/post-purchase)

Tool 11: loyalty_agent

Purpose

Awards loyalty points to the customer after a successful online purchase.

DB Operations

```
# Step 1: Get current loyalty points
customer = customer_crud.get_by_id(session, customer_id)

# Step 2: Get order items to calculate points earned
cart_items = cart_crud.filter(session, {"customer_id": customer_id})

# Step 3: Each product has a loyalty_points field -> sum them
points_earned = sum(product.loyalty_points for product in order_products)

# Step 4: Update customer loyalty balance
customer_crud.update(session, customer_id, {
    "loyalty_points": current_points + points_earned
})
```

When Called

Automatically after fulfillment_agent in online flow ONLY. Not called for offline purchases.

Tool 12: post_purchase_agent

Purpose

Provides post-purchase services including tracking, feedback, and customer support information.

Parameters

- order_id: str
- customer_id: str

Returns

```
{
  "order_tracking_link": "...",
  "feedback_form_link": "...",
  "customer_support_contact": "...",
  "return_exchange_policy": "7 days return/exchange"
}
```

When Called

Final step in online flow, after loyalty_agent. Not called for offline purchases.

Tools List Registration

```
tools = [
    customer_info_agent,      # Tool 1
    inventory_agent,          # Tool 2
    recommendation_agent,     # Tool 3
    upsell_agent,             # Tool 4
    add_to_cart_agent,        # Tool 5
    view_cart_agent,          # Tool 6
    remove_from_cart_agent,   # Tool 7
    choose_channel_agent,     # Tool 8
    payment_agent,            # Tool 9
    fulfillment_agent,        # Tool 10
    loyalty_agent,            # Tool 11
    post_purchase_agent       # Tool 12
]
```

6. Agent Graph & Orchestration (LangGraph)

File: backend/intelligent_agent_graph.py

State Definition (AgentState)

```
class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]  # Conversation
    customer_id: str
    selected_product: Dict[str, Any] | None
    inventory_status: str | None
    purchase_channel: str | None       # "online" or "offline"
    order_id: str | None
    payment_completed: bool
    loyalty_points: int
    cart_items: List[Dict[str, Any]]    # Products in cart
```

Graph Structure

```
StateGraph(AgentState)
    |-- add_node("agent", agent_node)     # LLM brain
    |-- add_node("tools", tool_node)       # Tool executor
    |-- set_entry_point("agent")           # Start here
    |-- add_conditional_edges("agent",
    |       should_continue,               # Route decision
    |       {"tools": "tools", "__end__": END})
    |-- add_edge("tools", "agent")         # Back to LLM
    compile()
```

Agent Node (The Brain)

This is the central decision-maker. It:
1. Creates the LLM: ChatOpenAI(model="gpt-4o-mini", temperature=0.1, streaming=True)
2. Binds all 12 tools: llm.bind_tools(tools)
3. Injects a massive system prompt containing:
   - Role definition ("helpful AI shopping assistant")
   - Conversation history awareness rules
   - Offline flow state tracking rules
   - Payment flow rules (two-phase)
   - Tool chaining rules (inventory -> recommendation)
   - The customer ID injected dynamically

Routing Logic (should_continue)

```
def should_continue(state: AgentState) -> Literal["tools", "__end__"]:
    last_message = state["messages"][-1]
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
```

```
        return "tools"     # LLM wants to call tools
    return "__end__"       # LLM is done, return response
```

Tool Executor (tool_node)

```
def tool_node(state: AgentState):
    outputs = []
    for tool_call in state["messages"][-1].tool_calls:
        tool_result = _execute_tool(tool_call)
        outputs.append(ToolMessage(
            content=tool_result,
            tool_call_id=tool_call["id"]
        ))
    return {"messages": outputs}
```

Run Agent Entry Point

```
async def run_agent(customer_id: str, query: str):
    graph = create_agent_graph()
    initial_state = {
        "messages": [HumanMessage(content=query)],
        "customer_id": customer_id,
        "selected_product": None,
        "purchase_channel": None,
        "order_id": None,
        "payment_completed": False,
        "loyalty_points": 0,
        "cart_items": [],
    }
    result = await graph.ainvoke(initial_state)
    return result
```

Complete Purchase Flows

Online Purchase Flow

```
User: "Show me hoodies"
  -> inventory_agent(search_query="hoodies")
  -> recommendation_agent(inventory_data)
  -> Display products

User: "Add P123, qty 2"
  -> add_to_cart_agent(product_id="P123", quantity=2)

User: "View cart"
  -> view_cart_agent(customer_id="C001")
  -> upsell_agent(customer_id="C001")  [automatic]

User: "Proceed to checkout"
  -> choose_channel_agent -> asks online/offline

User: "Online"
  -> payment_agent(customer_id)  [Phase 1: show methods]

User: "UPI"
  -> payment_agent(customer_id, payment_method="upi")  [Phase 2]
  -> fulfillment_agent(purchase_type="online")
  -> loyalty_agent(order_id, customer_id)
  -> post_purchase_agent(order_id, customer_id)
```

Offline Purchase Flow

```
User: "Show me hoodies"
  -> inventory_agent(search_query="hoodies")
  -> recommendation_agent(inventory_data)

User: "Add P123"
  -> add_to_cart_agent(...)

User: "Proceed offline"
  -> inventory_agent(product_id="P123")  [shows store list]

User: "Store ABC"
  -> fulfillment_agent(purchase_type="offline", store_name="ABC")
  -> END (no loyalty/post-purchase for offline)
```

7. All API Endpoints (Detailed)

File: backend/main.py

7.1 Root / Health Check

Method	Path	Description
GET	/	Returns {"message": "Hello world"} - Health check

7.2 Product Endpoints

Method	Path	Description
GET	/productdatabase/	Lists all products with full details, total count
GET	/productdatabase/{id}	Single product by product_id

7.3 Customer Endpoints

Method	Path	Description
GET	/api/customer/{id}	Get customer profile
GET	/api/customer/{id}/orders	Get customer order history
PUT	/api/customer/{id}	Update customer data

7.4 Store Endpoints

Method	Path	Description
GET	/api/stores	List all stores
GET	/api/stores/{id}	Single store details

7.5 Cart Endpoints

Method	Path	Description
GET	/api/cart/{customer_id}	Get customer active cart
POST	/api/cart	Add item to cart
PUT	/api/cart/update-quantity	Update item quantity in cart
PUT	/api/cart/update-size	Update item size in cart
DELETE	/api/cart/{order_id}	Remove item from cart
POST	/api/cart/apply-loyalty	Apply loyalty points discount
POST	/api/cart/unapply-	

loyalty

Restore loyalty points (undo discount)

7.6 Loyalty Points Logic (Cart Endpoints)

Apply Loyalty (/api/cart/apply-loyalty)

```
# Deducts loyalty points from customer balance
# Points used = min(cart_total, customer.loyalty_points)
# 1 point = Rs.1 discount
# Updates customer.loyalty_points in DB
```

Unapply Loyalty (/api/cart/unapply-loyalty)

```
# Restores points back to customer
new_points = current_points + points_to_restore
# Updates customer.loyalty_points in DB
```

7.7 Chat Endpoints

Method	Path	Description
POST	/api/chat	Main chat endpoint (REST) - sends message to agent
WS	/ws/chat	Real-time WebSocket chat with timeline updates
POST	/api/kiosk/chat	Store kiosk chat endpoint

POST /api/chat - Deep Dive

```
Request: {"customer_id": "C001", "message": "Show me hoodies"}

Flow:
1. Save user message to chat DB
2. Get recent conversation history (limit=1)
3. Check price band fast-path
   - If "products between 1500-2000" -> bypass LLM
   - Directly call inventory_agent + recommendation_agent
4. Normal path: result = await run_agent(customer_id, query)
5. Parse result messages for tool JSON responses
6. Extract timeline messages
7. Save assistant response to chat DB
8. Return response with agent_calls and timeline
```

Price Band Fast-Path

```
def _looks_like_price_band_query(text: str) -> bool:
    t = text.lower()
    # Skip cart/checkout keywords
    if re.search(r"\b(add|buy|purchase|checkout)\b", t):
        return False
    # Must mention price keyword + number
    if re.search(r"\b(between|above|under)\b", t) and re.search(r"\d", t):
        return True
    # Direct range: "1500-2000"
    if re.search(r"\b\d[\d,]*\s*-\s*\d[\d,]*\b", t):
```

```
            return True
        return False
```

7.8 Order History Endpoints

Method	Path	Description
GET	/api/orders/{customer_id}	Get all orders for customer
POST	/api/orders	Create new order

7.9 Admin Dashboard Endpoints

Method	Path	Description
GET	/api/admin/dashboard	Get admin analytics data
GET	/api/admin/transactions	Store keeper transactions
GET	/api/admin/stores	Store management data

7.10 Other Endpoints

Method	Path	Description
GET	/api/distance/{id}	Get distances from customer to all stores
POST	/api/feedback	Submit customer feedback
GET	/api/feedback/{id}	Get customer feedback
GET	/api/chat/history/{id}	Get conversation history
DELETE	/api/chat/history/{id}	Clear conversation history
GET	/api/payload/{id}	Get stored payload
POST	/api/payload	Save payload
GET	/webhook	WhatsApp webhook verification
POST	/webhook	Receive WhatsApp messages

8. WebSocket Real-time Communication

Endpoint: ws/chat (backend/main.py)

Message Types Sent to Client

Type	Purpose
timeline_start	An agent tool started executing
timeline_log	Log message from agent execution
timeline_complete	Agent tool finished successfully
timeline_error	Agent tool errored
text_delta	Streaming text chunk from AI response
final_response	Complete parsed response data (JSON)
run_end	All processing complete

Client Message Format

```
{
  "type": "chat",
  "request_id": "unique-id",
  "customer_id": "C001",
  "message": "Show me hoodies"
}
```

Server Streaming Sequence (Example)

```
{"type": "timeline_start", "agent": "Inventory Agent", "status": "in_progress"}
{"type": "timeline_log", "tool": "inventory_agent", "text": "Searching..."}
{"type": "timeline_complete", "tool": "inventory_agent", "status": "completed"}
{"type": "timeline_start", "agent": "Recommendation Agent"}
{"type": "timeline_complete", "tool": "recommendation_agent"}
{"type": "text_delta", "delta": "Here are some "}
{"type": "text_delta", "delta": "hoodie recommendations..."}
{"type": "final_response", "data": {...}}
{"type": "run_end"}
```

Helper Functions

- _ws_stream_text() - Sends text in small chunks (24 chars) for typewriter streaming effect
- _summarize_tool_start() - Human-readable description of tool call start
- _summarize_tool_result() - Human-readable description of tool result
- _extract_selected_store_name() - Parses store name from user message

9. Chat Database & History

File: backend/chat_database.py

This is a SEPARATE SQLite database (chat_history.db) for conversation persistence, isolated from the main business database.

Schema

```
CREATE TABLE conversations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id TEXT NOT NULL,
    session_id TEXT NOT NULL,
    role TEXT NOT NULL,          -- "user" or "assistant"
    content TEXT NOT NULL,
    agent_calls TEXT,            -- JSON array of tools called
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
)
```

Functions

Function	Purpose
init_database()	Creates conversations and payload tables
save_message()	Stores a message with timestamp and agent_calls
get_conversation_history()	Retrieves recent messages chronologically
clear_conversation_history()	Deletes all messages for a customer
get_conversation_stats()	Returns message counts and session info

10. Store Kiosk Flow

File: backend/store_kiosk_graph.py

A separate LangGraph for in-store kiosk interactions. More structured and linear compared to the main chatbot. Used when customers interact at physical store kiosks.

Kiosk Nodes

Node	Purpose
welcome_node	Greets customer, identifies them
kiosk_add_to_cart_agen	

t

Adds products to cart in-store

payment_prompt_node	Asks for payment
payment_method_node	Processes payment
fulfillment_node	Completes in-store purchase
loyalty_node	Awards loyalty points
post_purchase_node	Post-purchase feedback

Endpoint: POST /api/kiosk/chat -> chat_with_kiosk()

11. WhatsApp Integration

File: backend/whatsapp.py

Functions

Function	Purpose
send_message()	Sends a WhatsApp message via Meta API
get_text_message_input ()	Formats text for WhatsApp API payload

Webhook Endpoints

- GET /webhook - Meta verification handshake (challenge-response)
- POST /webhook - Receives incoming WhatsApp messages, processes via agent, responds

12. Security

File: backend/security.py

- signature_required - Decorator that validates WhatsApp webhook signatures using HMAC-SHA256
- Ensures incoming webhook requests are genuinely from Meta/WhatsApp

13. Data Import Pipeline

File: backend/utility/import_data.py

Imports CSV data from backend/data/ directory into SQLite tables:

Function	Source CSV	Target Table
import_customers()	customer_data.csv	customers
import_products()	product_data.csv	products
import_stores()	outlate_data.csv	stores
import_order_history()	order_history.csv	orderhistory
import_store_keeper()	store keeper data.csv	storekeeprtxn
import_admin_dashboard()	admin_dashboard.csv	admindashboard

Execution order matters - foreign key relationships require customers & products first, then orders.

14. Migration Scripts

Folder: backend/migrations/

Script	Purpose
add_quantity_column.py	Adds quantity column to products table
add_customer_type_columns.py	Adds customer_type and type_justification to customers
add_discount_column.py	Adds discount column
add_quantity_to_cart.py	Adds quantity to cart table
migrate_cart_schema.py	Schema migration for cart restructuring
populate_random_quantities.py	Fills random stock quantities (--seed for reproducibility)
fix_store_keeper_fk.py	Fixes foreign key in store keeper transactions

15. Configuration

File: backend/config.py - Loads configuration from environment variables

Environment Variables (.env.example)

```
OPENAI_API_KEY=your_openai_api_key_here
HOST=0.0.0.0
PORT=8000
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
LLM_TEMPERATURE=0.1
LLM_MODEL=gpt-4o-mini
```

16. Key Design Patterns Summary

1. Universal CRUD

One generic class for all DB operations, reducing boilerplate code across all models.

2. Tool-calling LLM

GPT-4o-mini decides which agent tools to invoke via OpenAI function calling. The LLM sees tool docstrings and schemas.

3. StateGraph (LangGraph)

Reactive loop: Agent -> Tools -> Agent -> Tools -> End. State is passed between nodes.

4. Two-phase Payment

Phase 1 shows payment options (no processing). Phase 2 actually processes the payment. Prevents accidental charges.

5. Dual Purchase Flow

Online flow (full pipeline with loyalty points) vs Offline flow (abbreviated, no loyalty/post-purchase).

6. In-memory Deduplication

Prevents showing same products twice using _SHOWN_PRODUCT_IDS_BY_CUSTOMER_AND_QUERY dictionary.

7. Price Band Fast-path

Bypasses LLM for simple price range queries, directly calling inventory + recommendation tools.

8. Real-time WebSocket Timeline

Shows which agents are working in real-time with streaming text deltas.

9. Separate Chat DB

Conversation persistence isolated from business data in a separate SQLite database.

10. Automatic Upselling

Random accessory suggestion triggered after cart view - intelligent cross-selling.

End of Documentation

EY Techathon Retail Agent

Multi-Agent AI Retail Assistant

Total Tools: 12 | Total Endpoints: 25+ | Database Tables: 10

Architecture: LangGraph StateGraph with GPT-4o-mini Orchestrator